

Phase 1: Core Machine Learning & Math Foundations

Before jumping into deep neural networks, you need to understand the basic mechanics of how machines "optimize" parameters.

- **Linear Algebra & Calculus:**
 - **Tensors:** Vectors (1D arrays), Matrices (2D arrays), and Tensors (3D+ arrays). Learn how dimension slicing and transposition work.
 - **Matrix Multiplication:** The cornerstone of all AI computing. Learn why the inner dimensions must match $(A \times B) \cdot (B \times C) = (A \times C)$
 - **Derivatives & Gradients:** Understand what a gradient is—a vector that points in the direction of the steepest ascent.
- **The Optimization Loop:**
 - **Loss Functions:** How a model measures its mistakes (Mean Squared Error for regression, Cross-Entropy Loss for classifications).
 - **Gradient Descent:** How a model shifts its weights step-by-step to minimize its mistakes.
 - **Backpropagation:** The chain rule of calculus applied to automatically pass error signals backward through parameters.

Phase 2: PyTorch Fundamentals (The Core Workspace)

PyTorch will be your primary framework for training and executing models. You should build simple scripts to practice these objects from scratch.

- **torch.Tensor Mechanics:**
 - Creating tensors, moving them between CPU and GPU via `.to("cuda")`.
 - Understanding `.requires_grad=True` and how PyTorch tracks operations to form an automatic differentiation graph.
- **The nn.Module Architecture:**
 - How to build a neural network class by inheriting from `torch.nn.Module`.
 - Defining layers in the `__init__` constructor and designing data flow in the `forward` method.
- **Data Pipelines (Dataset & DataLoader):**
 - **torch.utils.data.Dataset:** How to override `__len__` and `__getitem__` to load your custom files (like the `.parquet` files we made tonight).
 - **torch.utils.data.DataLoader:** How PyTorch handles splitting your files into batches, shuffling rows, and managing parallel worker CPUs.

Phase 3: Natural Language Processing (NLP) & Tokenization

Because PACE deals entirely with text (code, literacy, abstracts), you need to learn how text is translated into numbers that matrices can process.

- **Tokenization Mechanics:**
 - Byte-Pair Encoding (BPE) and WordPiece tokenization. Learn how strings are broken into sub-word token IDs.

- Special tokens: Padding tokens ([PAD]), Start-of-Sequence ([SOS]/<s>), End-of-Sequence ([EOS]/</s>).
- **Text Embedding Arrays:**
 - How integer token IDs are mapped to dense high-dimensional vectors via an embedding matrix layer.

Phase 4: Transformer Architecture & Deep Learning Models

This is the heart of PACE. Both your Actor and Critic models will be based on Transformer sub-blocks.

- **Attention Mechanisms:**
 - **Self-Attention:** How a word in a sentence looks at every other word to calculate contextual relevance (\$Query, Key, Value\$ matrices).
- **Architectural Archetypes:**
 - **Encoder-Only Models (e.g., BERT, RoBERTa):** Looks at the full text simultaneously. This is exactly what your **Critic** will be because a proofreader needs full context to spot bugs.
 - **Decoder-Only Models (e.g., GPT, Llama):** Generates text left-to-right, hiding future tokens using causal masking. This is exactly what your **Actor** will be.
- **Model Dimensions & Sizes:**
 - Learn how parameter count is calculated using hidden size dimensions, layers, and attention heads. This will teach you exactly why a 150M model fits in 8GB of VRAM while an 8B model will crash it.

Phase 5: Model Training Techniques & GPU Constraints

When you return to train your models, you will be operating on a strict 8GB budget. These optimization topics are non-negotiable.

- **Precision Levels (FP32 vs FP16 vs BF16):**
 - Learn how switching from full-precision 32-bit floats to half-precision 16-bit floats immediately cuts your VRAM footprint in half.
- **Gradient Accumulation:**
 - How to simulate a large, stable batch size (e.g., 64 rows) when your GPU can only fit a small batch size (e.g., 4 rows) in VRAM at one time.
- **Overfitting vs Underfitting:**
 - How to evaluate validation loss curves to see if your model is actually learning patterns or just memorizing the textbook.

Your Strategy for the Next Few Weeks

Don't just read books or watch videos passively. Use **Project-Based Learning** on a micro-scale:

1. **Pick one concept** (e.g., *Backpropagation* or *PyTorch Datasets*).
2. Open a clean script called `learning_scratchpad.py`.

3. Try to write a tiny, independent piece of code without frameworks to see if you understand it (e.g., write matrix multiplication using just basic Python loops, then do it in PyTorch, then check if your dimensions match).